

HPC Learning Guide

Pearson Correlation Recommender System

EC7207 High Performance Computing

Group 11

Parallelisation Versions Covered:

- | | |
|----------------------|-------------------------------------|
| 1. Serial Baseline | Algorithm foundation |
| 2. OpenMP | Shared-memory, fork-join |
| 3. Pthreads | Manual thread management |
| 4. MPI | Distributed-memory, message passing |
| 5. CUDA | GPU massively parallel |
| 6. Hybrid MPI+OpenMP | Two-level parallelism |

May 21, 2026

Contents

Introduction	2
1 Serial Baseline & Algorithm Foundation	4
1.1 Why a Serial Baseline?	4
1.2 The 5-Phase Algorithm	4
1.3 Pearson Correlation Formula	4
1.4 Prediction Formula	5
1.5 Key Constants	5
2 OpenMP — Shared-Memory Parallelism	6
2.1 Fork-Join Model	6
2.2 Directives Used in This Project	6
2.3 Schedule Comparison	6
2.4 Thread-Private Buffer Pattern (Phase 4)	6
2.5 Why <code>omp_get_wtime()</code> not <code>clock()</code> ?	7
3 Pthreads — Manual Thread Management	8
3.1 Key Difference from OpenMP	8
3.2 Core Pattern	8
3.3 Ceiling Division for Work Partitioning	8
3.4 No Mutex Needed	9
4 MPI — Distributed-Memory Parallelism	10
4.1 Core Concept: Private Memory Spaces	10
4.2 Initialisation	10
4.3 Key Collectives	10
4.4 Parallelisation Per Phase	11
5 CUDA — GPU Massively Parallel Computing	12
5.1 CPU vs. GPU	12
5.2 Thread Hierarchy	12
5.3 The Three Kernels	12
5.4 Memory Flow	12
5.5 Timing GPU Code	12
6 Hybrid MPI+OpenMP — Two-Level Parallelism	14
6.1 Motivation	14
6.2 Thread-Safe MPI Initialisation	14
6.3 Two-Level Work Division	14

6.4	Two-Level Reduction (Phase 5)	14
6.5	Benchmarked Configurations	15
	Summary Comparison	16

Introduction

Project Overview

This project implements a **User-Based Collaborative Filtering Recommender System** using Pearson Correlation Similarity, parallelised with five different HPC techniques.

The algorithm in one sentence: Given a sparse matrix of user ratings (1–5 stars), predict what rating each user would give to items they haven’t seen, using the ratings of the most similar users as a guide.

Performance Metrics

Metric	Definition
Speedup	$S(p) = T_{\text{serial}}/T_{\text{parallel}}(p)$
Efficiency	$E(p) = S(p)/p$ — fraction of ideal speedup
MAE	$\frac{1}{ T } \sum_{(u,i) \in T} \text{pred}(u,i) - \text{actual}(u,i) $
Sim checksum	Sum of all N^2 similarity values — cross-version sanity check

Output Tags (Parsed by Benchmark Script)

```
[Data]   Users: 1000 | Items: 1000 | Sparsity: 70% | Test ratings: 29866
[Timing] Similarity matrix   : 1.3549 s
[Timing] Prediction phase    : 7.0277 s
[Timing] Total (sim+pred)     : 8.3826 s
[Check]  Sim-matrix checksum: 942.387323
[Eval]   MAE on test set     : 1.2574
[Eval]   RMSE on test set    : 1.4579
```

(Actual serial-baseline output, 1000x1000, on the RTX 3050 reference machine.)

Amdahl’s Law — The Fundamental Limit

$$S_{\max}(p) = \frac{1}{f + (1-f)/p} \xrightarrow{p \rightarrow \infty} \frac{1}{f}$$

f = fraction of time that *cannot* be parallelised (data generation, evaluation). Here the *algorithmic* serial fraction is tiny ($f \approx 0.3\%$), but on this 8-core laptop the achievable

speedup is capped by memory bandwidth and reduced all-core clocks, giving an *effective* $f \approx 3.8\times$ ($S_{\max} \approx 26\times$). In practice the best CPU result is $\approx 6.3\times$ at 8 workers, while the GPU (CUDA) reaches $\approx 38\times$.

How the Files Relate

Source file	Topic covered in this guide
<code>serial/serial_recommender.c</code>	Chapter 1
<code>openmp/openmp_recommender.c</code>	Chapter 2
<code>pthread/pthreads_recommender.c</code>	Chapter 3
<code>mpi/mpi_recommender.c</code>	Chapter 4
<code>cuda/cuda_recommender.cu</code>	Chapter 5
<code>hybrid/hybrid_recommender.c</code>	Chapter 6

Chapter 1

Serial Baseline & Algorithm Foundation

1.1 Why a Serial Baseline?

Before parallelising anything, every HPC project must have a correct serial implementation serving three roles:

1. **Correctness reference:** every parallel MAE must match within ± 0.001
2. **Speedup denominator:** $S = T_{\text{serial}}/T_{\text{parallel}}$
3. **Algorithm clarity:** understand the logic before adding concurrency

1.2 The 5-Phase Algorithm

Phase	Operation	Complexity
1	Data Generation	$O(N \times M)$
2	User Means	$O(N \times M)$
3	Similarity Matrix	$O(N^2 \times M)$ — Bottleneck
4	Predictions	$O(N^2 \times M)$
5	MAE Evaluation	$O(T)$

1.3 Pearson Correlation Formula

$$\text{sim}(u, v) = \frac{\sum_{i \in I_{uv}} (R_{u,i} - \bar{R}_u)(R_{v,i} - \bar{R}_v)}{\sqrt{\sum_{i \in I_{uv}} (R_{u,i} - \bar{R}_u)^2} \times \sqrt{\sum_{i \in I_{uv}} (R_{v,i} - \bar{R}_v)^2}}$$

where I_{uv} = items rated by *both* u and v , and $\bar{R}_u$ = mean rating of user u .

Result range: $[-1, +1]$. $+1$ = always agree, 0 = unrelated, -1 = always disagree.

1.4 Prediction Formula

$$\text{pred}(u, i) = \bar{R}_u + \frac{\sum_{k \in K} \text{sim}(u, k) \cdot (R_{k,i} - \bar{R}_k)}{\sum_{k \in K} \text{sim}(u, k)}$$

K = top-20 most similar neighbours who rated item i with positive similarity.

1.5 Key Constants

```

1 #define DEFAULT_USERS 1000 /* rows in ratings matrix */
2 #define DEFAULT_ITEMS 1000 /* columns */
3 #define SPARSITY 0.70f /* 70% of entries are unrated */
4 #define TOP_K 20 /* max neighbours for prediction
   */
5 #define SEED 42 /* fixed RNG seed (same data every
   run) */
6 #define TEST_RATIO 0.10f /* 10% held out for MAE */

```

Chapter 2

OpenMP — Shared-Memory Parallelism

2.1 Fork-Join Model

OpenMP uses compiler directives (`#pragma omp`) to automatically manage a team of threads that share the same address space.

2.2 Directives Used in This Project

Phase	Directive	Schedule
2 (means)	<code>#pragma omp parallel for</code>	<code>static</code>
3 (sim)	<code>#pragma omp parallel for</code>	<code>dynamic, 4</code>
4 (pred)	<code>#pragma omp parallel + for</code>	<code>dynamic, 2</code>
5 (MAE)	<code>#pragma omp parallel for reduction(+:err)</code>	<code>static</code>

2.3 Schedule Comparison

Schedule	When to use	Overhead
<code>static</code>	Equal work per iteration	None
<code>dynamic,C</code>	Unequal/unpredictable work	Queue lookup

Phase 2 uses `static` (all rows scan exactly `N_ITEMS` items). Phases 3 & 4 use `dynamic` (work varies with sparsity and triangle shape).

2.4 Thread-Private Buffer Pattern (Phase 4)

```
1 #pragma omp parallel          /* fork threads */
2 {
3     SimPair *nbrs = malloc(N_USERS * sizeof(SimPair)); /*
4     private per thread */
5     #pragma omp for schedule(dynamic, 2)
```



```
5   for (int u = 0; u < N_USERS; u++) { /* distributed */ }  
6   free(nbrs);  
7 }
```

2.5 Why `omp_get_wtime()` not `clock()`?

`clock()` sums CPU time across all threads ($4 \times 1\text{s} = 4\text{s}$ for 4 threads each working 1 second). `omp_get_wtime()` measures actual wall-clock elapsed time (1s). Speedup requires wall-clock time.

Chapter 3

Pthreads — Manual Thread Management

3.1 Key Difference from OpenMP

Pthreads requires explicit thread creation, work partitioning, and joining. OpenMP hides all of this behind compiler directives.

3.2 Core Pattern

```
1  /* Thread function */
2  void *thread_fn(void *arg) {
3      ThreadArgs *a = (ThreadArgs *)arg;
4      int start, end;
5      work_range(N_USERS, a->tid, a->nthreads, &start, &end);
6      /* ... process rows [start, end) ... */
7      return NULL;
8  }
9
10 /* Launch in main() */
11 for (int t = 0; t < N_THREADS; t++) {
12     args[t] = (ThreadArgs){t, N_THREADS};
13     pthread_create(&threads[t], NULL, thread_fn, &args[t]);
14 }
15 for (int t = 0; t < N_THREADS; t++)
16     pthread_join(threads[t], NULL); /* wait = phase barrier */
```

3.3 Ceiling Division for Work Partitioning

```
1 void work_range(int total, int tid, int nthreads, int *start, int
   *end) {
2     int chunk = (total + nthreads - 1) / nthreads;
3     *start = tid * chunk;
4     *end    = min(*start + chunk, total);
5 }
```

Ensures all N items are covered with no gaps and no overlaps even when N is not divisible by the thread count.

3.4 No Mutex Needed

Our code avoids mutexes entirely because each thread writes to a disjoint range of indices. Mutexes would serialise access and eliminate the speedup benefit.

Chapter 4

MPI — Distributed-Memory Parallelism

4.1 Core Concept: Private Memory Spaces

Each MPI rank is an independent process with its own private RAM. Data must be explicitly sent and received using MPI functions — no shared pointers.

4.2 Initialisation

```
1 MPI_Init(&argc, &argv);
2 MPI_Comm_rank(MPI_COMM_WORLD, &mpi_rank); /* 0-based rank ID */
3 MPI_Comm_size(MPI_COMM_WORLD, &mpi_size); /* total ranks */
4 /* ... */
5 MPI_Finalize(); /* must be last MPI call */
```

4.3 Key Collectives

Call	Effect
MPI_Allgatherv	Each rank contributes a slice; every rank receives the complete assembled result. Used for <code>user_mean</code> and <code>sim_matrix</code> .
MPI_Reduce	Combines one value per rank (e.g. partial error sum) to rank 0. Used for MAE.
MPI_Barrier	All ranks wait until every rank reaches the barrier. Used to bracket timed sections.

4.4 Parallelisation Per Phase

Phase	Local work	Communication
1 (data gen)	Each rank runs <code>srand(42)</code> independently	None
2 (means)	Each rank computes its rows	<code>MPI_Allgatherv</code>
3 (sim)	Each rank fills all columns for its rows	<code>MPI_Allgatherv</code>
4 (pred)	Each rank predicts for its users only	None
5 (MAE)	Each rank sums its partial error	<code>MPI_Reduce</code>

Chapter 5

CUDA — GPU Massively Parallel Computing

5.1 CPU vs. GPU

GPU has thousands of simple cores; ideal for data-parallel work where the same operation applies to many independent elements. Our N^2 independent Pearson computations are a textbook GPU workload.

5.2 Thread Hierarchy

Thread: smallest unit, own registers, unique `threadIdx`

Block: up to 1024 threads sharing fast shared memory (`__syncthreads`)

Grid: all blocks for one kernel launch, scheduled across all SMs

5.3 The Three Kernels

Kernel	Grid	Thread maps to
<code>kernel_user_means</code>	1D	One thread per user
<code>kernel_similarity</code>	2D	One thread per (u, v) pair
<code>kernel_predictions</code>	2D	One thread per (u, item) pair

5.4 Memory Flow

CPU RAM $\xrightarrow{\text{cudaMemcpy (H2D)}}$ GPU DRAM $\xrightarrow{\text{kernel}}$ GPU DRAM $\xrightarrow{\text{cudaMemcpy (D2H)}}$ CPU RAM

5.5 Timing GPU Code

Use CUDA events (not `clock_gettime`) because kernel launches are *asynchronous*:

```
1 cudaEventRecord(ev_start);  
2 kernel<<<grid,block>>>(...);  
3 cudaEventRecord(ev_stop);
```

```
4 cudaEventSynchronize(ev_stop);  
5 float ms; cudaEventElapsedTime(&ms, ev_start, ev_stop);
```

Chapter 6

Hybrid MPI+OpenMP — Two-Level Parallelism

6.1 Motivation

Real clusters have multi-core nodes connected by a network.

- **Pure MPI:** intra-node communication is wasteful (cores share RAM)
- **Pure OpenMP:** cannot cross node boundaries
- **Hybrid:** MPI between nodes, OpenMP within nodes

6.2 Thread-Safe MPI Initialisation

```
1 int provided;  
2 MPI_Init_thread(&argc, &argv, MPI_THREAD_FUNNELED, &provided);  
3 /* MPI_THREAD_FUNNELED: MPI only called from main thread */
```

6.3 Two-Level Work Division

Level	Technology	Grain
Coarse	MPI	Rows assigned to different ranks
Fine	OpenMP	Threads share rows within a rank

Total workers = $P \times T$, MPI messages = P (not $P \times T$).

6.4 Two-Level Reduction (Phase 5)

1. OpenMP `reduction(+:local_err, local_cnt)`: private per-thread copies merged at barrier
2. MPI `Reduce`: per-rank totals summed to rank 0

6.5 Benchmarked Configurations

Config	$P \times T$	Notes
Hybrid_2x4	2 ranks, 4 threads	Balanced
Hybrid_4x2	4 ranks, 2 threads	More MPI
Hybrid_8x1	8 ranks, 1 thread	Equivalent to pure MPI
Hybrid_1x8	1 rank, 8 threads	Equivalent to pure OpenMP

Summary Comparison

Version	Memory	Parallelism	Best for	Communication
Serial	Shared	None	Baseline	None
OpenMP	Shared	Fork-join	Single machine	None
Pthreads	Shared	Manual threads	Full control	None
MPI	Distributed	Processes	Multi-node	Messages
CUDA	Host+GPU	1000s of threads	Data-parallel	PCIe transfer
Hybrid	Distributed	MPI×OMP	Large clusters	MPI only

Measured Speedup Order (this project; 8 CPU workers, CUDA on RTX 3050)

CUDA (×38) > OpenMP (×6.3) > Pthreads (×5.7) > MPI (×5.1) > Hybrid 8×1 (×4.9) >
Serial